Assignment 1: Analysis of SVD for Fine-Tuning LLMs

Big Data Analytics Techniques and Applications (Fall 2026) | Student ID: 315581033 (邱泓崴) | Code (public GitHub repository): https://github.com/Rewrite-StarrySky/BigData2026-Assignment1

Setup. PiSSA [1] fine-tuning of Llama-3.2-1B on the first 25k MetaMathQA examples (24,975 used; 25 dropped because the 512-token limit cut off the whole response), all 112 target matrices (q/k/v/o/gate/up/down in 16 layers, exact float32 SVD), 1 epoch, AdamW with learning rate 2e-5 (3% warm-up, cosine decay, no weight decay), effective batch 128 (196 updates), FP16, seed 42; evaluation with greedy decoding, max 256 new tokens, on the first 1,000 questions of the GSM8K and MATH test splits (accuracy = correct / 1,000); Kaggle 2×T4. Every number below comes from a single run. SE of an accuracy = $\sqrt{(p(1-p)/1000)}$ (1.2–1.4 pts on GSM8K, 0.6–0.7 pts on MATH); z = difference / SE of the difference (unpaired, so conservative: all models answer the same questions). "Unparsed" = GSM8K: the output contains no number at all; MATH: no answer marker ("answer …", "####" or `\boxed{}`) in the output, counted as wrong.

Question 1 (1%): Accuracy vs. target rank

(a) Accuracy of the original Llama-3.2-1B on GSM8K and MATH. (b) Fine-tuning with target rank 16, 64 and 256 (component = default). (c) Observations and explanations on the trends with different target ranks.

Accuracy	Original LLM	Fine-Tuned LLM (RANK = 16)	Fine-Tuned LLM (RANK = 64)	Fine-Tuned LLM (RANK = 256)
GSM8K	0.035	0.157	0.204	0.274
MATH	0.001	0.038	0.046	0.054
Trainable parameters	0	11.3M (0.9%)	45.1M (3.5%)	180.4M (12.%)
Unparsed outputs (GSM8K / MATH)	15 / 956	0 / 543	0 / 473	0 / 456
Last-update train loss†	–	0.455	0.414	0.375

† Loss of the last update only, i.e. the mean of its last 2 micro-batches (15 examples; the same 15 in every run): a noisy, weak indicator.

(c) Observations and explanations

- **Fine-tuning helps substantially.** GSM8K rises from 0.035 to 0.157–0.274 (4.5×–7.8×), MATH from 0.001 to 0.038–0.054. On MATH the original model gives no answer marker in 956/1,000 outputs (which alone caps its accuracy at 4.4%); after fine-tuning on MetaMathQA, whose responses end with "The answer is: …", this falls to 456–543, so part of the gain is learning to finish with a parseable answer. On GSM8K only 15 original outputs contain no number yet only 35 are correct, so there the baseline problem is wrong numbers, not missing ones (we did not inspect generations to say why).
- **Accuracy increases monotonically with rank on both datasets.** GSM8K: +4.7 pts for 16→64 (z = 2.7) and +7.0 pts for 64→256 (z = 3.7); MATH: +0.8 (z = 0.9) and +0.8 pts (z = 0.8), 16→256: +1.6 pts (z = 1.7). The GSM8K trend is clear and has not saturated at r = 256; the MATH steps are each within about one SE, so that trend is only suggestive.
- **Explanation.** A larger rank gives more trainable parameters (11.3M / 45.1M / 180.4M = 0.9% / 3.5% / 12.7% of all parameters incl. adapters) and starts from a larger principal subspace, leaving less of W in the frozen residual: for q_proj, ranks 16 / 64 / 256 hold 15–35% / 36–51% / 70–76% of the energy in layers 0 / 7 / 15 (Q4). The last-update loss also falls (0.455 → 0.414 → 0.375) but is only a 15-example average. These are plausible mechanisms consistent with the data; the runs do not isolate them.
- **GSM8K improves far more than MATH.** Even among MATH outputs that contain an answer only 8.3–9.9% are correct, so MATH is much harder for a 1B model. The MATH gain from r = 16 to 256 splits into two similar factors: outputs with a marker 457 → 544 (×1.19) and accuracy among them 8.3% → 9.9% (×1.19). *Hypothesis, not verified:* the 456–543 MATH outputs still without a marker are solutions cut off by the 256-token limit (generations were not saved, so this was not checked).

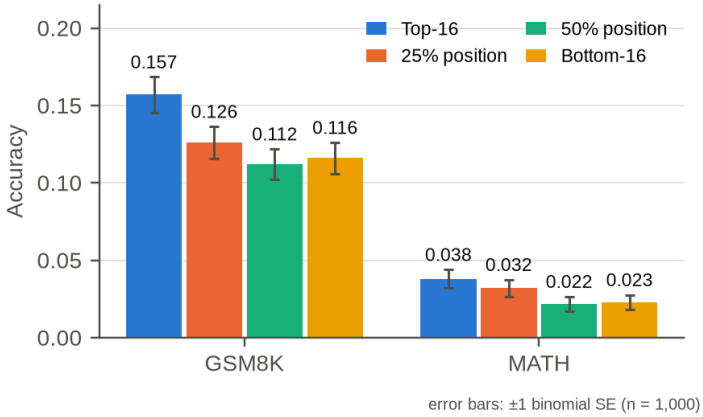
Question 2 (1%): Training time vs. target rank

	Fine-Tuned LLM (RANK = 16)	Fine-Tuned LLM (RANK = 64)	Fine-Tuned LLM (RANK = 256)
Training Time (min.)	**66.12**	**71.43**	**96.90**
Relative to RANK = 16	1.00×	1.08×	1.47×
Adapter / dense weights (112 matrices)	1.2%	4.6%	18.5%

- **Time grows with rank, but sub-linearly.** 4× the rank (16→64) costs +5.3 min (+8%); 16× (16→256) costs +30.8 min (+47%) although the trainable parameters grow 16×. The slope is 0.11 / 0.13 min per rank unit (16→64 / 64→256); the line through r = 16 and 256 extrapolates to ≈64 min at r = 0 (arithmetic on three single runs, not a fitted model).
- **Explanation.** Each layer computes $y = xW_{res}^T + (xB^T)A^T$. The frozen W_{res} matmuls (forward plus the activation-gradient backward; no weight gradient) cost the same at every rank, while the adapter adds $r(d_{in} + d_{out})$ multiply-adds per token, i.e. 1.2% / 4.6% / 18.5% of the 0.97B dense weights (about twice that in the backward pass). Counting these matmuls predicts only ≈26% more time at r = 256 than at r = 16, versus the measured +47%. The excess is not explained by FLOP counting; *possible causes (not profiled)*: the adapter branch runs in FP32 outside autocast while the frozen path runs in FP16 (T4: 8.1 vs. 65 TFLOPS FP32 vs. mixed precision [2]), skinny matmuls, and optimizer / DataParallel handling of 16× more trainable parameters.
- **Notes.** The one-time SVD initialisation of the 112 matrices takes 143–151 s (2.4–2.5 min) in all six runs and is not included above. The four rank-16 runs of Q3 train for 65.8–66.3 min, so cost depends on the rank, not on which singular values are selected.

Question 3 (1%): Which singular values to fine-tune (rank = 16)

Windows of 16 consecutive singular values: Top-16 (values 1–16, = “default”), 25% (values n/4+1…n/4+16), 50% (n/2+1…n/2+16) and Bottom-16 (n = 2,048, or 512 for k/v).



error bars: ±1 binomial SE (n = 1,000)

	Top-16	25%	50%	Bottom-16
GSM8K	0.157	0.126	0.112	0.116
MATH	0.038	0.032	0.022	0.023
MATH outputs with answer marker	457	411	385	396
Train loss†	0.455	0.495	0.497	0.507
Time (min.)	66.1	65.8	66.3	66.1

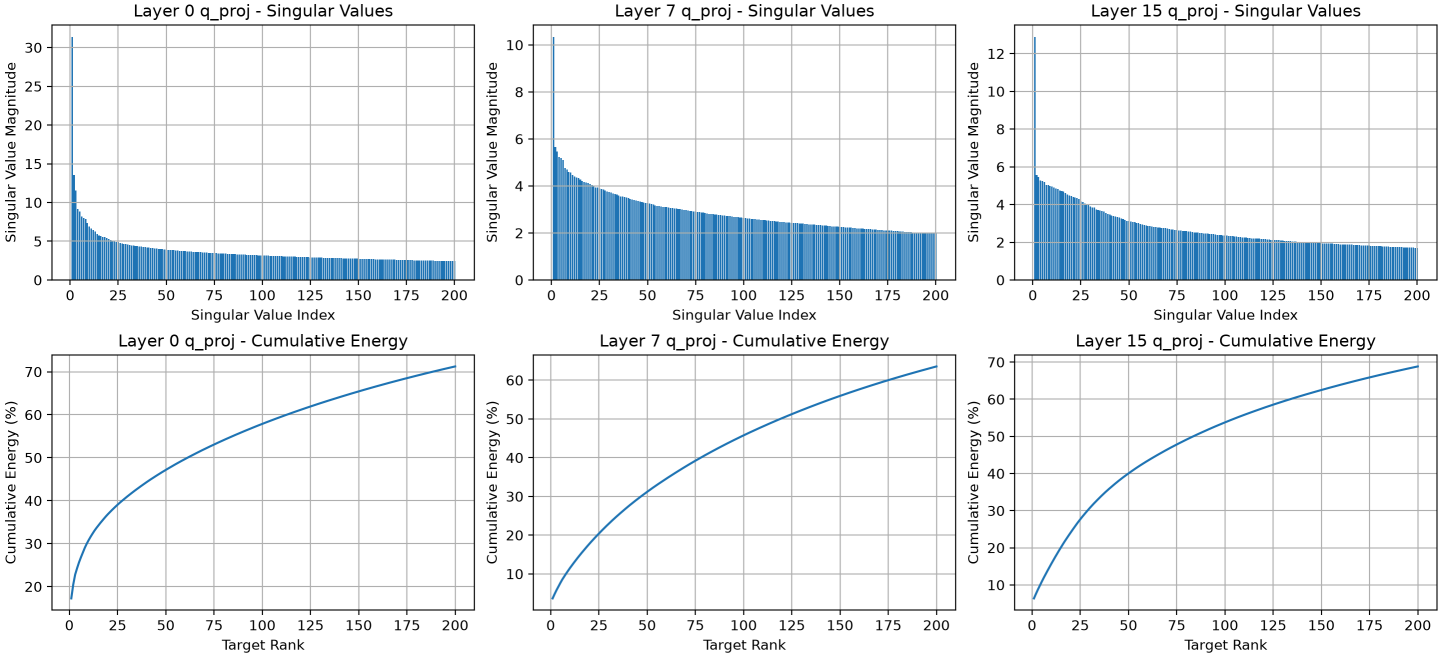
(b) Observations and explanations

- **Top-16 is best on both datasets, but clearly so only on GSM8K.** GSM8K 0.157 vs. 0.126 / 0.112 / 0.116 (+3.1 / +4.5 / +4.1 pts, z = 2.0 / 3.0 / 2.7); MATH 0.038 vs. 0.032 / 0.022 / 0.023 (+0.6 / +1.6 / +1.5 pts, z = 0.7 / 2.1 / 2.0), so the gap to the 25% window is within noise. The other three windows lie within 1.4 pts (GSM8K) and 1.0 pt (MATH) of each other (z ≤ 1.4); their order (25% > Bottom > 50%) is not read as a trend. Fewer MATH outputs end with an answer marker for the other windows (411 / 385 / 396 vs. 457).
- **Why the principal components win.** All four runs start from the same function ($W_{res} + AB = W$ up to a relative error ≤ 2.1e-04), so only *which* directions are trainable differs. The PiSSA paper argues that principal singular vectors are “the directions in which the matrix W has the most significant stretching or impact”, so tuning them lets the model “fit the training data faster and better”, while LoRA’s noise-and-zero initialisation gives gradients that “could be small and uninformative for a long time” [1, Sec. 1]. Windows from the middle or end of the spectrum have smaller singular values (in q_proj their indices are ≥ 513, so $\sigma \le \sigma_{200}$ = 1.7–2.4, versus σ_1 = 10–31), so A and B ($\propto \sigma^{1/2}$) start closer to zero, somewhat like LoRA’s zero initialisation. *Our own first-order reasoning:* $\Delta W = \Delta A \cdot B + A \cdot \Delta B$ scales with the entries of A and B, i.e. with $\sigma^{1/2}$. The paper’s own comparison of principal, medium and minor components on three 7B models also finds lower training loss and higher GSM8K and MATH accuracy for principal ones [1, App. F]. The last-update loss (0.455 vs. 0.495–0.507) points the same way but is only a 15-example average.
- **Every variant still beats the original model** (GSM8K 0.035, MATH 0.001), even Bottom-16: MATH outputs with an answer marker rise from 44 to 385–457 for every window, suggesting that every variant picks up the answer format of the training data (an interpretation; generations were not inspected).

Question 4 (2%): SVD of q_proj in the original Llama-3.2-1B (no fine-tuning)

(a, b) SVD of the 2048 × 2048 q_proj weight (float32); energy of rank r = $\Sigma_{i\le r}\sigma_i^2 / \Sigma_{all}\sigma_i^2$. (c) First 200 singular values: magnitudes (top row, bar charts) and cumulative energy (bottom row, line charts), as produced by the Q4 notebook.

	Layer 0	Layer 7	Layer 15
Minimum target rank for ≥ 50% energy	**62**	**120**	**84**
Cumulative energy at r = 16 / 64 / 200 / 256 (%)	34.8 / 50.6 / 71.2 / 76.4	15.4 / 35.9 / 63.5 / 70.1	21.0 / 44.7 / 68.8 / 74.3
σ_1 (share of total energy)	31.4 (17.2%)	10.3 (3.7%)	12.9 (6.4%)
σ_{200} / smallest σ	2.43 / 7e-06	1.98 / 8e-05	1.70 / 5e-05



(d) Observations

- **q_proj is only weakly low-rank.** A steep head (σ_1 = 31.4 / 10.3 / 12.9 = 12.9× / 5.2× / 7.6× σ_{200}) is followed by a slow decay: 62 / 120 / 84 singular values (3.0% / 5.9% / 4.1% of 2,048) are needed for 50% of the energy and the first 200 hold only 63.5–71.2%. The curves are concave (descending singular values) and still rising at r = 200. The smallest singular values (7e-06–8e-05) are tiny but non-zero; in float32 they are only accurate to about $\varepsilon\cdot\sigma_1 \approx$ 1e-6–4e-6, so we do not interpret them further.
- **Concentration differs by layer and is not monotonic in depth.** Among the three layers, layer 0 is the most concentrated (σ_1 alone holds 17.2% of the energy), layer 7 the flattest (3.7%) and layer 15 lies in between (6.4%). *Extra analysis of all 16 layers* (not required; extra_q4_all_layers.ipynb) shows that layer 7 is typical of the interior: layers 1–13 all need ≥ 114 singular values (maximum: layer 2, 184), while layers 0, 14, 15 need 62, 74, 84. The PiSSA authors suspect that “middle layers in a model tend to have more even eigenvalue distributions due to redundancy and insufficient training” [1, App. G]; our measurement is consistent with this for q_proj but does not test the cause.

Layer	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
Min. rank, ≥ 50%	62	174	184	172	162	144	139	120	118	114	118	144	146	152	74	84

- **Implications.** (i) One global rank keeps different fractions per layer: r = 64 keeps 51% / 36% / 45% in layers 0 / 7 / 15 (28–51% over all 16 layers), so interior layers need a larger rank for the same fidelity. (ii) Ranks 16 / 64 / 256 cover 12–35% / 28–51% / 59–76% of q_proj’s energy (all layers), in line with Q1 where accuracy kept improving up to r = 256. (iii) In Q3 only the Top-16 window comes from the steep head of the spectrum. These are indicative only: Q4 covers q_proj, whereas Q1/Q3 adapt all seven matrix types in 16 layers.

[1] F. Meng, Z. Wang, M. Zhang. PiSSA: Principal Singular Values and Singular Vectors Adaptation of Large Language Models. NeurIPS 2024 (arXiv:2404.02948). [2] NVIDIA T4 specifications, nvidia.com/en-us/data-center/tesla-t4. All numbers are taken from the executed notebooks in the repository (bigdata-hw1-q1/q3/q4.ipynb, extra_q4_all_layers.ipynb).